Conjugate Gradient vs. Gradient Descent
Implementation Notes for the Reduced Optimal Control Problem

Fachseminar

February 22, 2026

Abstract

This note explains how the fixed-step gradient descent (GD) and the conjugate gradient (CG) solvers are implemented in the accompanying code base for the reduced, strictly convex quadratic optimal control problem. In particular, we clarify the "optimization as a linear system" viewpoint: the unique minimizer of the quadratic reduced functional solves a symmetric positive definite (SPD) linear system, which can be solved efficiently by CG. We also summarize why CG is typically much faster than GD for SPD quadratics.

1 Problem and Discrete Reduced Form

We consider the reduced objective in control coefficients $\mathbf{u} \in \mathbb{R}^n$,

$$F(\mathbf{u}) = \tfrac{1}{2}\,(A^{-1}B\mathbf{u} - \mathbf{y}_d)^\top M\,(A^{-1}B\mathbf{u} - \mathbf{y}_d) + \tfrac{\beta}{2}\,\mathbf{u}^\top M_U\,\mathbf{u}, \tag{1}$$

where A is the discrete stiffness matrix (with homogeneous Dirichlet treatment), M is the state L^2 mass matrix, M_U is the control mass matrix, and B is the control-to-state coupling (restriction of the source to the tagged subdomain ω). The regularization parameter satisfies $\beta > 0$.

Expanding the quadratic yields the unconstrained quadratic program

$$\min_{\mathbf{u}\in\mathbb{R}^n} \tfrac{1}{2}\,\mathbf{u}^\top Q\,\mathbf{u} - \mathbf{b}^\top\mathbf{u} + \text{const}, \qquad Q := B^\top A^{-1} M A^{-1} B + \beta M_U, \qquad \mathbf{b} := B^\top A^{-1} M\,\mathbf{y}_d. \tag{2}$$

Under the assumptions used in the project (in particular $\beta > 0$), Q is SPD and the minimizer is unique.

Control inner product used in the code. The methods are written in the control-space inner product induced by M_U,

$$\langle \mathbf{a}, \mathbf{b}\rangle_U := \mathbf{a}^\top M_U\,\mathbf{b}, \qquad \|\mathbf{a}\|_U := \sqrt{\langle \mathbf{a}, \mathbf{a}\rangle_U}. \tag{3}$$

Accordingly, the gradient used by the algorithms is the Riesz-represented gradient in this metric.

2 "Optimization as a Linear System" and CG

For a quadratic functional of the form (2), the first-order optimality condition is linear:

$$\nabla F(\mathbf{u}_*) = \mathbf{0} \quad\Longleftrightarrow\quad Q\,\mathbf{u}_* = \mathbf{b}. \tag{4}$$

This is the key point behind the professor's remark: instead of iterating an optimization method, we can compute the minimizer by solving the SPD linear system (4). CG is specifically designed for large SPD systems and typically converges in far fewer iterations than GD.

2.1 Matrix-free $Q\,\mathbf{v}$ in the code

The implementation avoids forming Q explicitly and only needs the matrix-vector product $\mathbf{v} \mapsto Q\,\mathbf{v}$. The code first implements the reduced Hessian action in the U-metric,

$$H\,\mathbf{v} = M_U^{-1} B^\top A^{-1} M A^{-1} B\,\mathbf{v} + \beta\,\mathbf{v}. \tag{5}$$

Multiplying by M_U gives

$$M_U\,(H\,\mathbf{v}) = (B^\top A^{-1} M A^{-1} B + \beta M_U)\mathbf{v} = Q\,\mathbf{v}. \tag{6}$$

In other words, the matvec is implemented as

$$Q\,\mathbf{v} \equiv M_U\,(H\,\mathbf{v}), \tag{7}$$

which is exactly what the Python code does when it defines a `LinearOperator` with `q_matvec(v) = MU @ hess_U(v)`.

2.2 Right-hand side b in the code

The vector $\mathbf{b} = B^\top A^{-1} M \mathbf{y}_d$ is computed by one adjoint-like solve:

$$\mathbf{z} := M\,\mathbf{y}_d, \tag{8}$$

$$A\,\mathbf{p} = \mathbf{z} \quad \text{(solve)}, \tag{9}$$

$$\mathbf{b} := B^\top \mathbf{p}. \tag{10}$$

This matches the routine used to build the CG right-hand side.

2.3 CG algorithm (as used)

CG is applied to the SPD system (4) with a matrix-free operator. In SciPy's interface, one provides only the action $\mathbf{v} \mapsto Q\mathbf{v}$. A concise description is:

Algorithm 1: Conjugate Gradient for the minimizer $\mathbf{u}_*$

Data: SPD operator Q via matvec, right-hand side $\mathbf{b}$, tolerance `rtol`
Result: Approximation $\mathbf{u}$ to the solution of $Q\mathbf{u} = \mathbf{b}$
Initialize $\mathbf{u}_0$ (SciPy default is $\mathbf{0}$);
$\mathbf{r}_0 \leftarrow \mathbf{b} - Q\mathbf{u}_0$;
$\mathbf{p}_0 \leftarrow \mathbf{r}_0$;
for $k = 0, 1, \ldots$ *until convergence* **do**
  $\alpha_k \leftarrow \dfrac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{p}_k^\top Q \mathbf{p}_k}$;
  $\mathbf{u}_{k+1} \leftarrow \mathbf{u}_k + \alpha_k\,\mathbf{p}_k$;
  $\mathbf{r}_{k+1} \leftarrow \mathbf{r}_k - \alpha_k\,Q\mathbf{p}_k$;
  $\beta_k \leftarrow \dfrac{\mathbf{r}_{k+1}^\top \mathbf{r}_{k+1}}{\mathbf{r}_k^\top \mathbf{r}_k}$;
  $\mathbf{p}_{k+1} \leftarrow \mathbf{r}_{k+1} + \beta_k\,\mathbf{p}_k$;
end

In exact arithmetic CG terminates in at most n iterations. In practice it converges much sooner when Q is reasonably well-conditioned (or after preconditioning).

3 Fixed-step Gradient Descent (GD) Implementation

GD is implemented as a first-order method that iterates

$$\mathbf{u}_{k+1} = \mathbf{u}_k - \alpha \nabla F(\mathbf{u}_k), \tag{11}$$

where the gradient is the U-Riesz gradient,

$$\nabla F(\mathbf{u}) = M_U^{-1} B^\top \mathbf{p} + \beta \mathbf{u}, \qquad A\mathbf{p} = M(\mathbf{y}(\mathbf{u}) - \mathbf{y}_d), \qquad A\mathbf{y}(\mathbf{u}) = B\mathbf{u}. \tag{12}$$

3.1 Step size choice $\alpha = 1/L$

For an L-Lipschitz gradient in the U-metric, a safe fixed step is $\alpha = 1/L$. The code estimates $L \approx \lambda_{\max}(H)$ by power iteration applied to the Hessian action $H\mathbf{v}$ and a U-Rayleigh quotient.

3.2 Stopping criterion and logging

The GD loop stops when $\|\nabla F(\mathbf{u}_k)\|_U \leq$ `tol` (or a maximum iteration count is reached). The implementation stores the gradient norms and objective values in a history dictionary for plotting/analysis.

4 Why CG Should Be Faster Than GD (for this problem)

Both GD and CG are iterative methods, but they exploit curvature very differently.

GD (fixed-step) rate. For a strongly convex quadratic with eigenvalues in $[m, L]$ (in the relevant metric), fixed-step GD with $\alpha = 1/L$ converges linearly with a factor approximately

$$\|\mathbf{u}_k - \mathbf{u}_*\| \lesssim \left(1 - \frac{m}{L}\right)^k \|\mathbf{u}_0 - \mathbf{u}_*\|. \tag{13}$$

If the condition number $\kappa := L/m$ is large, then $1 - m/L = 1 - 1/\kappa$ is close to 1 and GD becomes slow.

CG rate. CG has a sharper dependence on the condition number. A standard bound in the Q-energy norm is

$$\frac{\|\mathbf{u}_k - \mathbf{u}_*\|_Q}{\|\mathbf{u}_0 - \mathbf{u}_*\|_Q} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}\right)^k. \tag{14}$$

The appearance of $\sqrt{\kappa}$ (instead of κ) explains why CG is typically much faster than GD on ill-conditioned SPD problems.

Interpretation for the reduced OC system. Here, $Q = B^\top A^{-1} M A^{-1} B + \beta M_U$ is SPD and often moderately to strongly ill-conditioned depending on mesh size and β. In such settings, CG (especially with preconditioning) is expected to reach a good accuracy in far fewer iterations than fixed-step GD.

5 Mapping to the Repository Code

The relevant implementations are:

- **GD:** `gd_fixed` in `Code/optimizers.py`. It uses $\alpha = 1/L$, logs $F(\mathbf{u}_k)$ and $\|\nabla F(\mathbf{u}_k)\|_U$, and stops at `tol`.

- **CG for $\mathbf{u}_*$:** `compute_u_star` in `Code/run_methods.py` (and similarly in `Code/mesh_independence.py`). It builds a matrix-free `LinearOperator` for Q, assembles $\mathbf{b} = B^\top A^{-1} M \mathbf{y}_d$, and calls `scipy.sparse.linalg.cg` with a relative tolerance. A callback is used to count iterations.

- **Hessian action:** `hess_U` in `Code/reduced_oc_model.py`, used both for L estimation and for the Q matvec in CG.

6 Practical Notes

Fair comparison. If one compares GD and CG fairly as solvers for (4), then GD corresponds to steepest descent on the same quadratic. CG usually wins by a large margin in iteration counts, but each iteration has a similar dominant cost here: applying Q once, which entails two PDE solves through the A^{-1} applications.

Preconditioning (not used here). If CG iteration counts still grow with mesh refinement, the natural next step is preconditioning (e.g., using M_U or a multigrid-type approximation). This can reduce κ dramatically.